

Reviewer Pack — Minimal Order of Galois Representations over Tseitin Circuit...

Entry 231c35be54ba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 14:33:14 UTC

Minimal Order of Galois Representations over Tseitin Circuit Size

Entry ID: 231c35be54ba **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 14:33:14 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Galois Representations) **Field B** (complexity object): Complexity Theory: Tseitin Circuit Size

Statement:

[‘For every Tseitin circuit C with n inputs and m clauses, the minimal order of a Galois representation $\phi: G \rightarrow GL_2(\mathbb{F}_p)$ associated with C is $\Theta(m^{\frac{1}{2}})$, where G is the group of permutations of the variables that preserve the circuit structure.’, ‘The minimal order of ϕ is defined as the smallest integer p such that $\phi(G) \subseteq SL_2(\mathbb{F}_p)$.’, ‘This conjecture implies that for circuits with a fixed number of inputs, the size of the Galois representation grows at least as fast as the square root of the number of clauses.’]

Rationale (proposer’s reasoning):

[“Galois representations provide a way to encode arithmetic properties of groups, and their minimal order can reveal information about the complexity of functions that permute group elements. The connection to Tseitin circuit size suggests that the complexity of these permutations might be related to the circuit’s structural properties.”, ‘The minimal order of a Galois representation is often difficult to compute directly, but its relation to the square root of the number of clauses in a Tseitin circuit offers a potential link between number theory and computational complexity that could lead to new insights.’]

Taxonomy category: Geometric Group Theory intersected with Circuit Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7cb46b4b4e7d4bc0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for 30 random Tseitin circuits with fixed n inputs and varying m clauses, the minimal order of the Galois representation ϕ scales as $\Theta(m^{\frac{1}{2}})$ with

an aggregate mean within $\pm 10\%$ of $m^{\{1/2\}}$ and no seed produces a deviation greater than 20% from this mean.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal order Galois representations Tseitin circuit AND Number Theory
- Galois representation order $\Theta(m^{\{1/2\}})$ Tseitin circuits complexity theory - order $\phi(G) \subseteq \text{SL}_2(\mathbb{F}_p)$ Tseitin circuits number theory

Top relevant hits considered: - [http://arxiv.org/abs/1611.09315v1] Oddness of residually reducible Galois representations - [http://arxiv.org/abs/2001.08287v3] Wild Galois representations: a family of hyperelliptic curves with large inertia image - [http://arxiv.org/abs/1407.5802v2] Galois representations and Galois groups over $\mathbb{Q}$ - [http://arxiv.org/abs/1603.02493v4] Clifford theory for glider representations - [http://arxiv.org/abs/0711.0795v4] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space - [http://arxiv.org/abs/1808.08676v3] Constraining the p-mode-g-mode tidal instability with GW170817 - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90\text{s}$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 5 # Number of inputs
    m_values = [10, 20, 30, 40] # Different numbers of clauses to test

    metric_name = "min_galois_representation_order"
    total_orders = []
    conjecture_holds = True
    counterexample = ""

    for m in m_values:
        circuit = generate_tseitin_circuit(n, m)
        G = construct_group_action(circuit)
```

```

order = find_minimal_order(G)
total_orders.append(order)

expected_order = m ** 0.5
if abs(order - expected_order) / expected_order > 0.2:
    conjecture_holds = False
    counterexample = f"m={m}, order={order}, expected={expected_order}"

mean_value = sum(total_orders) / len(total_orders)
std_value = (sum((x - mean_value) ** 2 for x in total_orders) / len(total_orders)) ** 0.5

return {
    "metric_name": metric_name,
    "metric_value": mean_value,
    "instances_tested": len(m_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

def generate_tseitin_circuit(n: int, m: int) -> list:
    inputs = [f"x{i}" for i in range(1, n + 1)]
    clauses = []

    # Generate literals and their negations
    literals = inputs + [f"~{x}" for x in inputs]

    # Generate random clauses
    for _ in range(m):
        clause = random.sample(literals, 2)
        if random.choice([True, False]):
            clause[0] = f"~{clause[0]}"
        if random.choice([True, False]):
            clause[1] = f"~{clause[1]}"
        clauses.append(clause)

    return inputs + clauses

def construct_group_action(circuit: list) -> set:
    inputs = circuit[:circuit.index(inputs[-1])]
    G = set()

    for perm in itertools.permutations(inputs):
        if is_valid_permutation(perm, circuit):
            G.add(perm)

    return G

def is_valid_permutation(perm: tuple, circuit: list) -> bool:
    inputs = circuit[:circuit.index(inputs[-1])]
    for clause in circuit[len(inputs):]:
        literals = [x.strip("~") for x in clause]
        if all(lit not in perm and f"~{lit}" not in perm for lit in literals):
            return False
    return True

def find_minimal_order(G: set) -> int:

```

```

p = 2
while True:
    found = False
    for g in G:
        if any(all((g[i] == j and g[j] != i) or (g[i] != j and g[j] == i) for i, j in enumerate(g))):
            found = True
            break
    if not found:
        return p
    p += 1

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = (sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c19b0081.py", line 83, in <module>
    trial_result = run_trial(seed)
                  ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c19b0081.py", line 52, in run_trial
    G = construct_group_action(circuit)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c19b0081.py", line 36, in construct_group_action
    for perm in itertools.permutations(inputs):
                          ~~~~~^~~~~~

```

NameError: name 'inputs' is not defined. Did you mean: 'input'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be evaluated. | next: Debug the test code to ensure it runs successfully and produces the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12667
2	preregistration	ollama_remote	glm4:latest	0	0	6198
3	novelty	ollama_remote	glm4:latest	0	0	4802
4	novelty	ollama_remote	glm4:latest	0	0	6856
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17875
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10191
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10389
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12268
9	judge	ollama_remote	glm4:latest	0	0	8792

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 90038 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/231c35be54ba.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/231c35be54ba.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/231c35be54ba.tar.gz` (if generated)